

PRAKTYKA PROGRAMOWANIA

Laboratorium 4

Wzorce Projektowe – Instrukcje i Implementacje

Most (Bridge)

Nazwa

Most (ang. Bridge)

Problem

Most jest wzorcem strukturalnym. Wyobraź sobie, że tworzysz aplikację graficzną obsługującą różne kształty (Koło, Kwadrat, Trójkąt) i różne platformy renderowania (SVG, Canvas, PDF). Naiwne podejście – jedna klasa na każdą kombinację – prowadzi do eksplozji klas:

```
KołoSVG, KołoCanvas, KołoPDF  
KwadratSVG, KwadratCanvas, KwadratPDF  
TrójkątSVG, TrójkątCanvas, TrójkątPDF  
→ 3 kształty × 3 platformy = 9 klas (i rośnie wykładniczo)
```

Wzorzec Most stosujemy wtedy, gdy:

- Chcemy uniknąć trwałego powiązania (przez dziedziczenie) między abstrakcją a jej implementacją – chcemy móc zmieniać je niezależnie.
- Mamy dwa ortogonalne (niezależne) wymiary zmienności, np. "co robię" (kształt) i "jak to robię" (platforma).
- Zmiana implementacji nie powinna wymagać modyfikacji kodu klientów ani klas abstrakcji.
- Chcemy móc podmieniać implementację w czasie działania programu (runtime).

Rozwiązanie – mechanizm działania

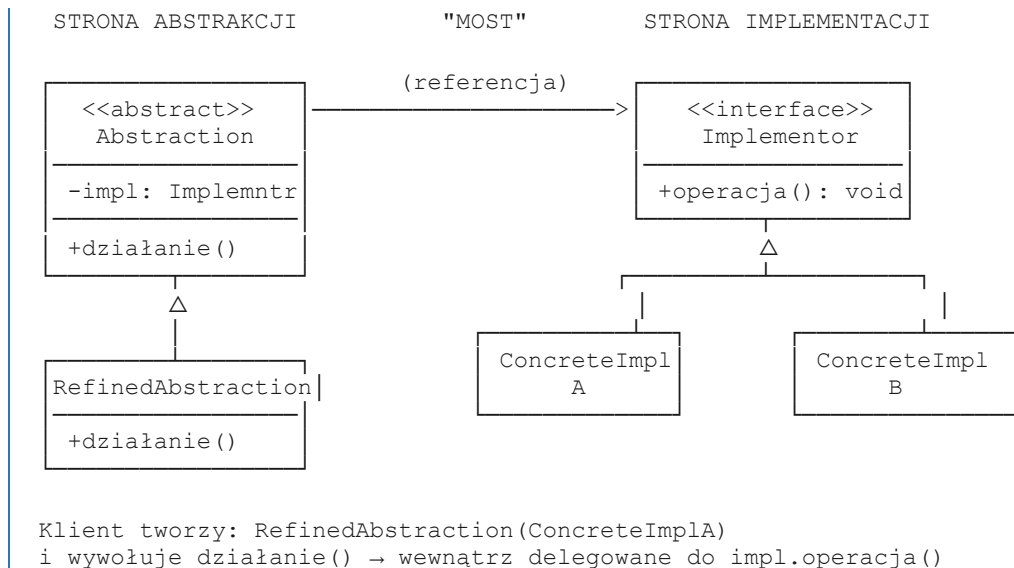
Wzorzec Most rozwiązuje problem przez zastąpienie dziedziczenia kompozycją. Zamiast jednej rozgałęziającej się hierarchii tworzymy dwie osobne, połączone "mostem" (referencją):

Struktura – cztery elementy wzorca

- Implementor – interfejs (klasa abstrakcyjna) definiujący operacje niskopoziomowe, które są specyficzne dla platformy/implementacji. Nie musi odpowiadać 1:1 interfejsowi Abstraction – może być bardziej prymitywny.

- ConcreteImplementor – konkretna klasa implementująca interfejs Implementor. Każda platforma/technologia to osobna klasa. Nie wie nic o klasach abstrakcji.
- Abstraction – klasa abstrakcyjna (wysoki poziom) opisująca "co". Przechowuje referencję do obiektu Implementor (to jest właśnie "most"). Deleguje niskopoziomową pracę do Implementora zamiast samodzielnie ją wykonywać.
- RefinedAbstraction – konkretna klasa dziedzicząca po Abstraction. Rozszerza lub specjalizuje zachowanie. Korzysta z odziedziczonej referencji do Implementora.

Schemat klas (UML)



Kluczowy mechanizm – delegacja przez referencję

Sercem wzorca jest to, że Abstraction NIE implementuje operacji samodzielnie – przekazuje wywołanie do obiektu Implementor trzymanego jako pole klasy:

```
klasa Abstraction:
    konstruktor(impl: Implementor):
        self._impl = impl          # <-- tu jest "most"

    metoda działanie():
        # Abstraction robi swoją logikę wysokiego poziomu...
        self._impl.operacja()      # ...i deleguje do Implementora

klasa RefinedAbstraction dziedziczy po Abstraction:
    metoda działanie():
        # może rozszerzać zachowanie
        self._impl.operacja()      # nadal korzysta z tego samego mostu
```

Implementor jest wstrzykiwany przez konstruktor (Dependency Injection). Dzięki temu można go podmienić w czasie działania programu bez zmiany klasy Abstraction.

Kroki implementacji

1. Zidentyfikuj dwa ortogonalne wymiary zmienności (np. kształt i platforma renderowania).
2. Stwórz interfejs Implementor z operacjami niskopoziomowymi (np. `rysuj_okrąg(x, y, r)`).
3. Zaimplementuj ConcreteImplementor dla każdej platformy/technologii (np. `SVGRenderer`, `CanvasRenderer`).
4. Stwórz abstrakcyjną klasę Abstraction, która przyjmuje Implementor w konstruktorze i przechowuje go jako pole prywatne.
5. Zaimplementuj metody Abstraction tak, by delegowały niskopoziomową pracę do Implementora.

6. Stwórz klasy `RefinedAbstraction` dziedziczące po `Abstraction` (np. `Koło`, `Kwadrat`).
7. Klient tworzy wybrany `ConcretImplementor`, przekazuje go do konstruktora `RefinedAbstraction` i używa tylko interfejsu `Abstraction`.

Przykład – kształty i platformy renderowania

Dwa wymiary zmienności: kształt (`Koło`, `Kwadrat`) i technologia rysowania (`SVG`, `Canvas`).
Liczba klas: $2 + 2 = 4$ zamiast $2 \times 2 = 4$ – zysk widoczny przy większej skali (np. 5 kształtów $\times$ 4 platformy = 9 klas zamiast 20).

Konsekwencje

Zalety

- Niezależna rozszerzalność obu hierarchii – nowy kształt nie wymaga zmian w `Rendererach` i odwrotnie.
- Podmiana implementacji w czasie działania – można zmienić `Implementor` bez rekompilacji `Abstraction`.
- Zasada otwarte/zamknięte – kod otwarty na rozszerzenia, zamknięty na modyfikacje.
- Ukrycie szczegółów implementacji przed klientem – klient używa tylko interfejsu `Abstraction`.

Wady

- Zwiększona złożoność kodu – konieczność stworzenia co najmniej 4 klas/interfejsów nawet dla prostego przypadku.
- Nadmiarowość dla prostych przypadków – jeśli implementacja nigdy się nie zmienia, wzorzec jest zbędny.
- Trudniejsze zrozumienie przepływu – delegacja przez referencję jest mniej oczywista niż dziedziczenie.